Sistema de Reservas de Hotel

Memoria del Proyecto Final

Asignatura: Base de Datos

Autores: Kelvin Del Castillo · Gabriel Galasso · Camila Vásquez

Fecha de entrega: Abril de 2026

Repositorio: https://github.com/GalaxyGG1/Software-Reservas

Índice

1. Introducción
2. Planteamiento del problema
3. Diseño de la base de datos 3.1. Diagrama Entidad-Relación 3.2. Modelo Relacional 3.3. Diccionario de datos 3.4. Decisiones de diseño
4. Sentencias SQL desarrolladas 4.1. DDL — Creación de la base 4.2. Restricciones y reglas de negocio 4.3. Procedimientos almacenados 4.4. Triggers 4.5. Vista de reportes 4.6. DML — Operaciones CRUD desde la aplicación
5. Integración con la aplicación
6. Conclusiones
7. Anexos

---

1. Introducción

El presente trabajo documenta el diseño, implementación y puesta en marcha de un **Sistema de Reservas de Hotel**, desarrollado como proyecto final de la asignatura de Base de Datos. El objetivo del proyecto es sustituir el sistema de archivos planos utilizado en la aplicación originalmente construida en la asignatura de Programación Orientada a Objetos por una solución persistente, transaccional y consistente, basada en un Sistema Gestor de Bases de Datos (SGBD) relacional.

La aplicación, construida en **C# con Windows Forms (.NET Framework 4.8)** y respaldada por **Microsoft SQL Server**, permite gestionar todas las operaciones del día a día de un hotel multi-sucursal: registro de clientes, control de habitaciones, reservas, check-in/check-out, facturación con impuestos configurables, servicios adicionales, promociones, mantenimiento, reportes gerenciales y administración de usuarios con control de acceso por roles.

Este documento describe paso a paso las decisiones tomadas en cada capa del sistema —desde el modelo conceptual hasta la integración con la aplicación—, las sentencias SQL desarrolladas y la experiencia adquirida durante el proceso.

---

2. Planteamiento del problema

2.1. Contexto

Un hotel mediano con varias sucursales necesita un sistema centralizado que permita a recepcionistas, administradores y personal de soporte gestionar la operación diaria sin depender de hojas de cálculo o archivos planos. El sistema previo, basado en archivos serializados, presentaba los siguientes problemas:

- **Concurrencia:** dos recepcionistas no podían operar simultáneamente sin pisarse cambios.
- **Integridad referencial:** no había forma de garantizar que una reserva apuntara a un cliente y a una habitación válidos.
- **Auditoría:** no se podía rastrear quién hizo qué y cuándo.
- **Reportes:** generar informes operativos requería abrir el archivo y procesarlo manualmente.
- **Escalabilidad:** el sistema no soportaba múltiples sucursales ni roles diferenciados.

2.2. Requisitos funcionales

El sistema debe permitir:

Módulo	Funcionalidad
Autenticación	Login con usuario y contraseña hasheada (SHA-256). Roles: ADMIN, RECEPCION, EMPLEADO, SOPORTE.
Clientes	CRUD completo, búsqueda por documento.
Habitaciones	CRUD por sucursal, control de estados (DISPONIBLE, OCUPADA, MANTENIMIENTO, LIMPIEZA).
Reservas	Crear/cancelar/confirmar reservas, validación de solapamiento de fechas.
Check-in / Check-out	Workflow controlado, generación automática de factura al checkout.
Facturación	Cálculo automático de subtotal, descuentos, impuestos (ITBIS configurable) y total.
Servicios adicionales	Catálogo y asociación a reservas.
Promociones	Descuentos porcentuales o por monto fijo, aplicables a reservas.
Mantenimiento	Registro de mantenimientos por habitación con estados y costos.
Reportes	Operativos y gerenciales (ocupación, RevPAR, ADR, GOPPAR).
Administración	Gestión de empleados y usuarios.
Auditoría	Registro automático de acciones críticas.

2.3. Requisitos no funcionales

- **Persistencia confiable** mediante un SGBD relacional con transacciones ACID.
- **Integridad referencial** declarada a nivel de la base de datos mediante claves foráneas.
- **Reglas de negocio** encapsuladas en `CHECK constraints`, triggers y procedimientos almacenados.
- **Seguridad** — contraseñas hasheadas; consultas parametrizadas para prevenir inyección SQL.
- **Configurabilidad** — parámetros del sistema (impuestos, moneda) almacenados en una tabla específica.

---

3. Diseño de la base de datos

3.1. Diagrama Entidad-Relación

El diagrama E/R completo se incluye en el **Anexo A** (`Diagrama_Hotel.drawio.pdf`).

Las entidades principales identificadas son:

- **Cliente** — persona que reserva habitaciones.
- **Empleado** — personal del hotel; puede tener una cuenta de usuario asociada.
- **Usuario** — credenciales de login asociadas a un empleado y a un rol.
- **Sucursal** — sede física del hotel.
- **Habitación** — tipificada por `TipoHabitacion`, ubicada en una sucursal y con un estado.
- **Reserva** — vincula cliente, habitación y empleado durante un rango de fechas.
- **Factura** — generada a partir de una reserva.
- **Pago** — uno o varios pagos cubren una factura.

Las entidades de soporte son: `Mantenimiento`, `CheckInOut`, `Notificacion`, `Rating`, `AuditLog` y `ConfiguracionSistema`.

3.2. Modelo Relacional

El modelo relacional se transformó siguiendo las reglas estándar:

- Las relaciones 1:N quedaron resueltas mediante claves foráneas (`Habitacion → Sucursal`, `Reserva → Cliente`, etc.).
- Las relaciones N:M se convirtieron en tablas asociativas con clave primaria compuesta:
- `ReservaServicio (IdReserva, IdServicioAdicional)`
- `ReservaPromocion (IdReserva, IdPromocion)`
- Los atributos multivaluados, como los pagos por factura, se modelaron como entidad débil (`Pago`).

El modelo relacional completo se entrega en el **Anexo B** (`Modelo Relacional - Hotel.xlsx`).

3.3. Diccionario de datos (resumen)

A continuación se listan las tablas principales con sus atributos clave. El diccionario completo se encuentra en `docs/DICCIONARIO_DATOS.md` del repositorio.

Roles

Atributo	Tipo	Restricciones
`IdRol`	INT	PK, IDENTITY
`NombreRol`	VARCHAR(20)	UNIQUE, NOT NULL
`Descripcion`	VARCHAR(100)	NULL

Usuarios

Atributo	Tipo	Restricciones
`IdUsuario`	INT	PK, IDENTITY
`IdEmpleado`	INT	FK → Empleado, NULL (cuentas técnicas)
`NombreUsuario`	VARCHAR(50)	UNIQUE, NOT NULL
`Clave`	VARCHAR(128)	NOT NULL (hash SHA-256)
`IdRol`	INT	FK → Roles, NOT NULL

Reserva

Atributo	Tipo	Restricciones
`IdReserva`	INT	PK, IDENTITY
`IdCliente`	INT	FK → Cliente
`IdHabitacion`	INT	FK → Habitacion
`IdEmpleado`	INT	FK → Empleado
`IdEstadoReserva`	INT	FK → EstadoReserva
`FechaEntrada`	DATE	NOT NULL
`FechaSalida`	DATE	NOT NULL, CHECK (> FechaEntrada)
`CantidadHuespedes`	INT	CHECK (> 0)
`Subtotal` , `Descuento` , `Total`	DECIMAL(10,2)	NOT NULL, CHECK (>= 0)

(Para el resto de tablas ver `HotelDB_Proyecto_Final.sql` y el archivo `DICCIONARIO_DATOS.md`.)

3.4. Decisiones de diseño

1. **Estados como tablas catálogo, no como CHECK.** Para `EstadoHabitacion` y `EstadoReserva` se optó por tablas catálogo (`EstadoHabitacion` , `EstadoReserva`) en lugar de un `CHECK constraint` con valores fijos. Esto permite agregar estados nuevos sin alterar la estructura.
2. **Estados volátiles como CHECK.** En cambio, en `Mantenimiento.Estado` , `Factura.Estado` y `CheckInOut.Estado` —cuyos valores son cerrados y rara vez cambian— se usó `CHECK constraint` por simplicidad.
3. **`CheckInOut` separado de `Reserva` .** Se modeló como entidad propia para registrar las fechas y empleados de check-in y check-out de forma independiente, permitir auditoría y soportar reservas que nunca lleguen a check-in (cancelaciones).

4. `AuditLog` **genérico.** Una sola tabla con campos `Entidad` y `IdEntidad` permite auditar cualquier tabla sin multiplicar tablas de log.
5. `ConfiguracionSistema` **clave-valor.** Permite cambiar parámetros (porcentaje de ITBIS, moneda, hora de check-in) sin redesplegar la aplicación.
6. **Hash SHA-256 para contraseñas.** Las contraseñas se almacenan hasheadas para evitar exposición en caso de fuga. La aplicación calcula el hash antes de enviarlo al SGBD.
7. **Triggers para sincronizar** `Habitacion.Estado`**.** Cuando una reserva pasa a CONFIRMADA, la habitación pasa a OCUPADA automáticamente; al cancelarse o finalizarse, regresa a DISPONIBLE.

---

4. Sentencias SQL desarrolladas

El script completo y comentado se encuentra en `Base de Datos/HotelDB_Proyecto_Final.sql`. A continuación se destacan las sentencias más relevantes.

4.1. DDL — Creación de la base

```sql
CREATE DATABASE HotelDB;
GO
USE HotelDB;
GO

CREATE TABLE dbo.Reserva
(
    IdReserva INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    IdCliente INT NOT NULL,
    IdHabitacion INT NOT NULL,
    IdEmpleado INT NOT NULL,
    IdEstadoReserva INT NOT NULL,
    FechaReserva DATETIME NOT NULL DEFAULT (GETDATE()),
    FechaEntrada DATE NOT NULL,
    FechaSalida DATE NOT NULL,
    CantidadHuespedes INT NOT NULL,
    Subtotal DECIMAL(10,2) NOT NULL DEFAULT (0),
    Descuento DECIMAL(10,2) NOT NULL DEFAULT (0),
    Total DECIMAL(10,2) NOT NULL DEFAULT (0),

    CONSTRAINT CK_Reserva_Fechas CHECK (FechaSalida > FechaEntrada),
    CONSTRAINT CK_Reserva_Huespedes CHECK (CantidadHuespedes > 0),
    CONSTRAINT FK_Reserva_Cliente
        FOREIGN KEY (IdCliente) REFERENCES dbo.Cliente(IdCliente),
    CONSTRAINT FK_Reserva_Habitacion
        FOREIGN KEY (IdHabitacion) REFERENCES dbo.Habitacion(IdHabitacion)
    -- ...
);
```

4.2. Restricciones y reglas de negocio

- **Unicidad de habitación por sucursal:** `UNIQUE (IdSucursal, Numero)` en `Habitacion`.
- **Documento único por cliente:** `UNIQUE (Documento)` en `Cliente`.
- **Validación de fechas en reservas:** `CHECK (FechaSalida > FechaEntrada)`.
- **No solapamiento de reservas:** validado en `sp_InsertarReserva` (no como CHECK porque requiere consultar otras filas).

4.3. Procedimientos almacenados

`sp_ValidarLogin`

Recibe usuario y hash de contraseña; devuelve datos del usuario si las credenciales coinciden y el usuario está activo.

`sp_InsertarReserva`

Encapsula la lógica de creación de reserva: 1. Valida que la fecha de salida sea posterior a la de entrada. 2. Verifica que la

habitación no tenga reservas solapadas en estados PENDIENTE/CONFIRMADA. 3. Calcula el subtotal usando `fn_CalcularNoches` y el precio de la habitación. 4. Inserta la reserva.

`sp_GenerarFactura`

Genera la factura de una reserva: 1. Si no se pasa `@PorcImpuesto`, lo lee de `ConfiguracionSistema.ITBIS_PORCENTAJE`. 2. Suma subtotal de hospedaje + servicios adicionales. 3. Resta descuentos de reserva + descuentos por promociones. 4. Calcula impuesto y total; inserta la factura.

```
CREATE OR ALTER PROCEDURE dbo.sp_GenerarFactura
    @IdReserva INT,
    @PorcImpuesto DECIMAL(5,2) = NULL
AS
BEGIN
    DECLARE @ItbisReal DECIMAL(5,2);

    IF @PorcImpuesto IS NULL
        SELECT @ItbisReal = CAST(Valor AS DECIMAL(5,2))
        FROM dbo.ConfiguracionSistema
        WHERE Clave = 'ITBIS_PORCENTAJE';
    ELSE
        SET @ItbisReal = @PorcImpuesto;

    -- ... cálculo de subtotal, descuentos e impuesto ...
END;
```

4.4. Triggers

`trg_Reserva_ActualizarEstadoHabitacion`

Sincroniza `Habitacion.IdEstadoHabitacion` automáticamente: - Reserva → CONFIRMADA ⇒ Habitación → OCUPADA - Reserva → CANCELADA o FINALIZADA ⇒ Habitación → DISPONIBLE

`trg_Mantenimiento_ActualizarEstadoHabitacion`

- Mantenimiento → PENDIENTE / EN PROCESO ⇒ Habitación → MANTENIMIENTO
- Mantenimiento → FINALIZADO ⇒ Habitación → DISPONIBLE

4.5. Vista de reportes

```
CREATE OR ALTER VIEW dbo.vw_RendimientoEmpleados
AS
SELECT
    e.IdEmpleado,
    e.Nombre + ' ' + e.Apellido AS Empleado,
    s.NombreSucursal,
    SUM(CASE WHEN er.NombreEstado <> 'CANCELADA' THEN 1 ELSE 0 END) AS TotalReservas,
    ISNULL(SUM(ISNULL(f.Total, 0)), 0) AS TotalFacturado
FROM dbo.Empleado e
INNER JOIN dbo.Sucursal s ON s.IdSucursal = e.IdSucursal
LEFT JOIN dbo.Reserva r ON r.IdEmpleado = e.IdEmpleado
LEFT JOIN dbo.EstadoReserva er ON er.IdEstadoReserva = r.IdEstadoReserva
LEFT JOIN dbo.Factura f ON f.IdReserva = r.IdReserva
GROUP BY e.IdEmpleado, e.Nombre, e.Apellido, s.NombreSucursal;
```

4.6. DML — Operaciones CRUD desde la aplicación

Las clases `*Data.cs` de la aplicación encapsulan las operaciones DML. Ejemplo de inserción de cliente desde C#:

```
string sql = @"
    INSERT INTO dbo.Cliente
        (Nombres, Apellidos, TipoDocumento, Documento, Telefono, Correo, Direccion)
    VALUES
        (@Nombres, @Apellidos, @TipoDocumento, @Documento, @Telefono, @Correo, @Direccion);";

using (SqlCommand cmd = new SqlCommand(sql, conn))
{
    cmd.Parameters.AddWithValue("@Nombres",       nombres);
    cmd.Parameters.AddWithValue("@Apellidos",     apellidos);
    cmd.Parameters.AddWithValue("@TipoDocumento", tipoDoc);
    cmd.Parameters.AddWithValue("@Documento",     documento);
    cmd.Parameters.AddWithValue("@Telefono",      telefono);
    cmd.Parameters.AddWithValue("@Correo",        correo);
    cmd.Parameters.AddWithValue("@Direccion",     direccion);
    cmd.ExecuteNonQuery();
}
```

Todas las consultas usan parámetros (`@nombre`) y nunca concatenación de strings, para prevenir inyección SQL.

---

5. Integración con la aplicación

La arquitectura de la aplicación sigue una separación en tres capas:

```
UI (Forms)
    ↓
Data Classes (*Data.cs)
    ↓
Database.cs  ← único punto de conexión
    ↓
SQL Server (HotelDB)
```

- **`Database.cs`** centraliza la cadena de conexión y el método `GetConnection()`. También define `HashPassword(string)` (SHA-256 hex en minúsculas) y `RegistrarAccion(...)` para escribir en `AuditLog`.
- **Las clases `*Data.cs`** (`AdministracionData`, `ReportesData`, `CheckInOutData`, etc.) contienen la SQL específica de cada módulo y exponen métodos a los formularios.
- **Los formularios `*Form.cs`** no contienen SQL — sólo interactúan con las clases `*Data.cs`. Cada formulario suscribe al evento `Theme.OnThemeChanged` para soportar modo claro/oscuro.

El acceso por roles se gestiona en `MainForm.ConfigurarSegunRol()`, que oculta o muestra los botones del menú lateral según `SesionUsuario.Rol`.

---

6. Conclusiones

El proyecto nos permitió aplicar de extremo a extremo el flujo completo de una solución basada en bases de datos relacionales: desde la abstracción del problema en un diagrama E/R, hasta la integración del SGBD con una aplicación de escritorio funcional.

Lecciones aprendidas

- **El diseño correcto del modelo paga dividendos.** Las primeras versiones del diagrama E/R no separaban `CheckInOut` de `Reserva`; al hacerlo, ganamos auditoría y cláridad en el flujo.
- **Las restricciones a nivel de BD son la red de seguridad.** Aunque la aplicación validaba fechas y montos, agregar `CHECK constraints` y `FOREIGN KEYS` a nivel del SGBD eliminó toda una clase de bugs por estados inconsistentes.
- **Procedimientos almacenados para lógica transaccional.** Inicialmente generábamos la factura desde C# con varias llamadas; trasladar la lógica a `sp_GenerarFactura` simplificó el código del cliente y centralizó las reglas.
- **Configuración como datos.** Mover el porcentaje de ITBIS a `ConfiguracionSistema` permitió que el sistema sobreviviera a cambios fiscales sin recompilar.

Dificultades enfrentadas

- **Migración de contraseñas en texto plano a SHA-256.** Tuvimos que escribir un script idempotente que sólo hashee filas que aún están en texto plano, detectando por longitud (`LEN(Clave) < 32`).
- **Sincronización de estados entre `Reserva` y `Habitacion`.** Implementar la lógica en triggers fue elegante pero requirió cuidado para que `INSERT` y `UPDATE` se manejaran correctamente.
- **Concurrencia de check-in/check-out.** El procedimiento envuelve check-out + generación de factura en una `SqlTransaction` para garantizar atomicidad.
- **Inspección dinámica de columnas.** En `AdministracionData.cs` se utiliza `sys.columns` para tolerar variaciones de nombres en la tabla `Roles` (`NombreRol`, `Rol`, `Nombre`), facilitando la portabilidad.

Grado de dificultad

El proyecto resultó **moderadamente desafiante**. La parte más entretenida fue diseñar el modelo de datos pensando en escenarios reales (multi-sucursal, mantenimiento, reportes gerenciales). La parte más exigente fue mantener consistencia entre las reglas de negocio implementadas en C# y las reglas declaradas en la BD —la decisión final fue **declarar primero a nivel de BD** y dejar que la aplicación dependiera de esa autoridad.

---

7. Anexos

- **Anexo A** — `Diagrama_Hotel.drawio.pdf` (Diagrama Entidad-Relación)
- **Anexo B** — `Modelo Relacional - Hotel.xlsx` (Modelo Relacional)
- **Anexo C** — `Base de Datos/HotelDB_Proyecto_Final.sql` (Script DDL/DML completo)
- **Anexo D** — `LEER.txt` (Instrucciones de instalación y ejecución)
- **Anexo E** — Capturas de pantalla de la aplicación (ver al final del documento).
- **Anexo F** — Repositorio de la aplicación: https://github.com/GalaxyGG1/Software-Reservas

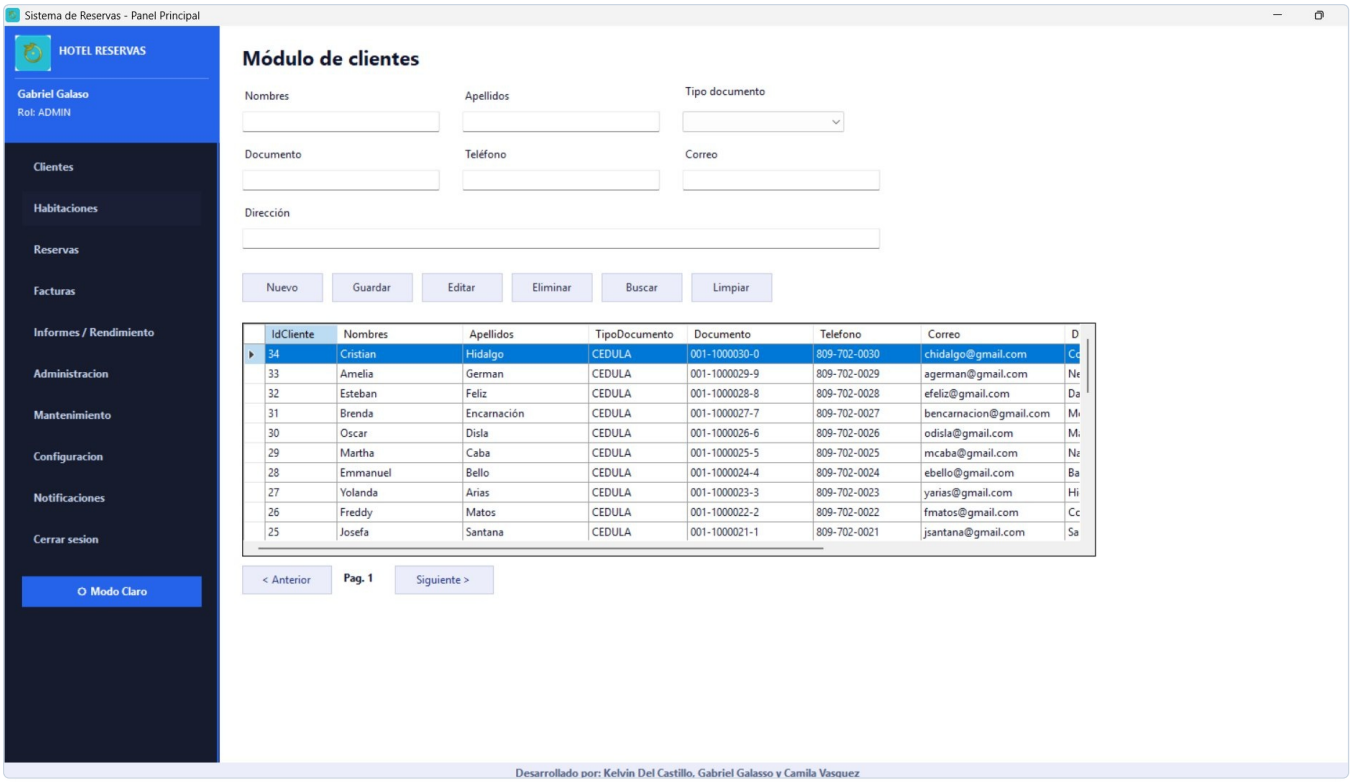
---

Anexo E — Capturas de pantalla de la aplicación

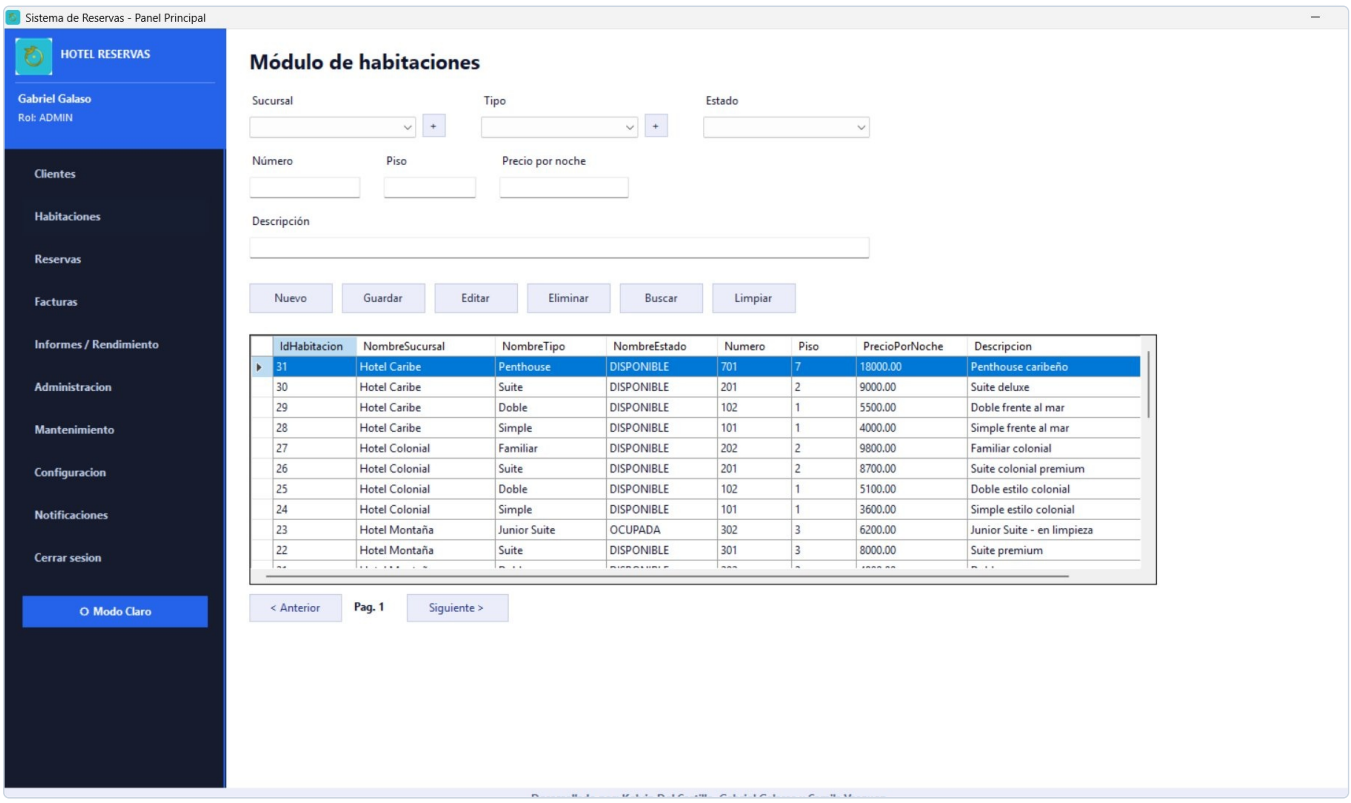
A continuación se incluyen capturas representativas de los principales módulos del sistema, ilustrando el flujo de uso desde el inicio de sesión hasta la administración y configuración.

Pantalla de login con modo oscuro/claro y diseño de bienvenida.

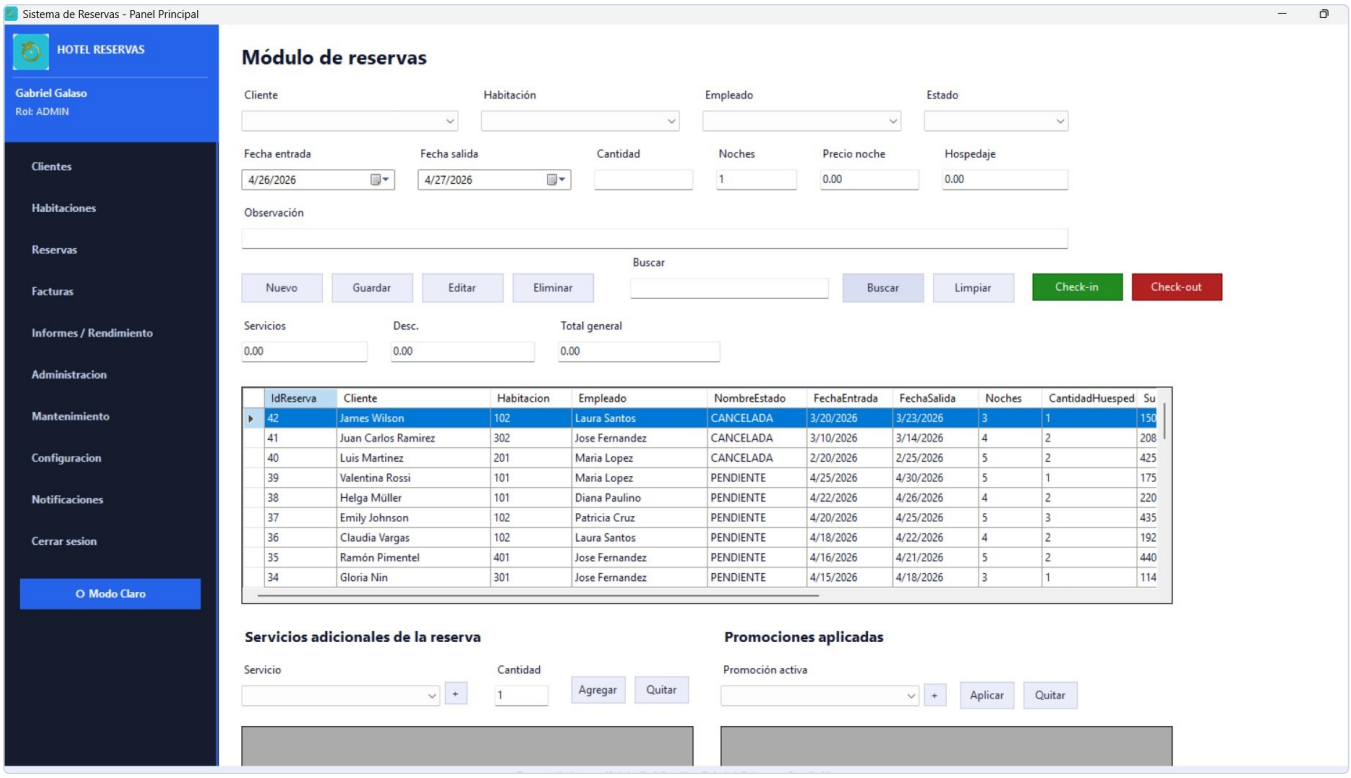
Dashboard operativo: reservas del día, ocupación actual, ingresos del mes y mantenimientos pendientes, con gráfico de habitaciones ocupadas.



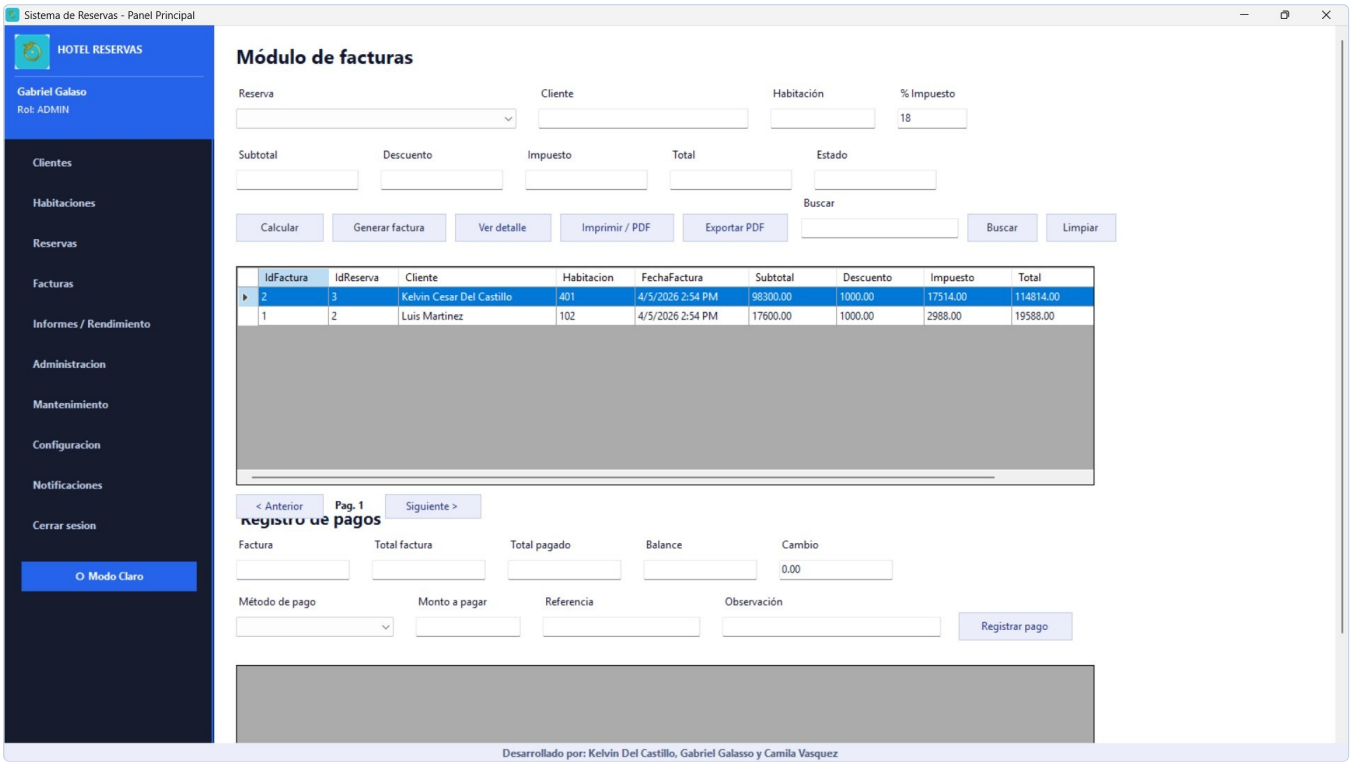
Módulo de clientes: formulario CRUD, búsqueda por documento y grilla paginada.



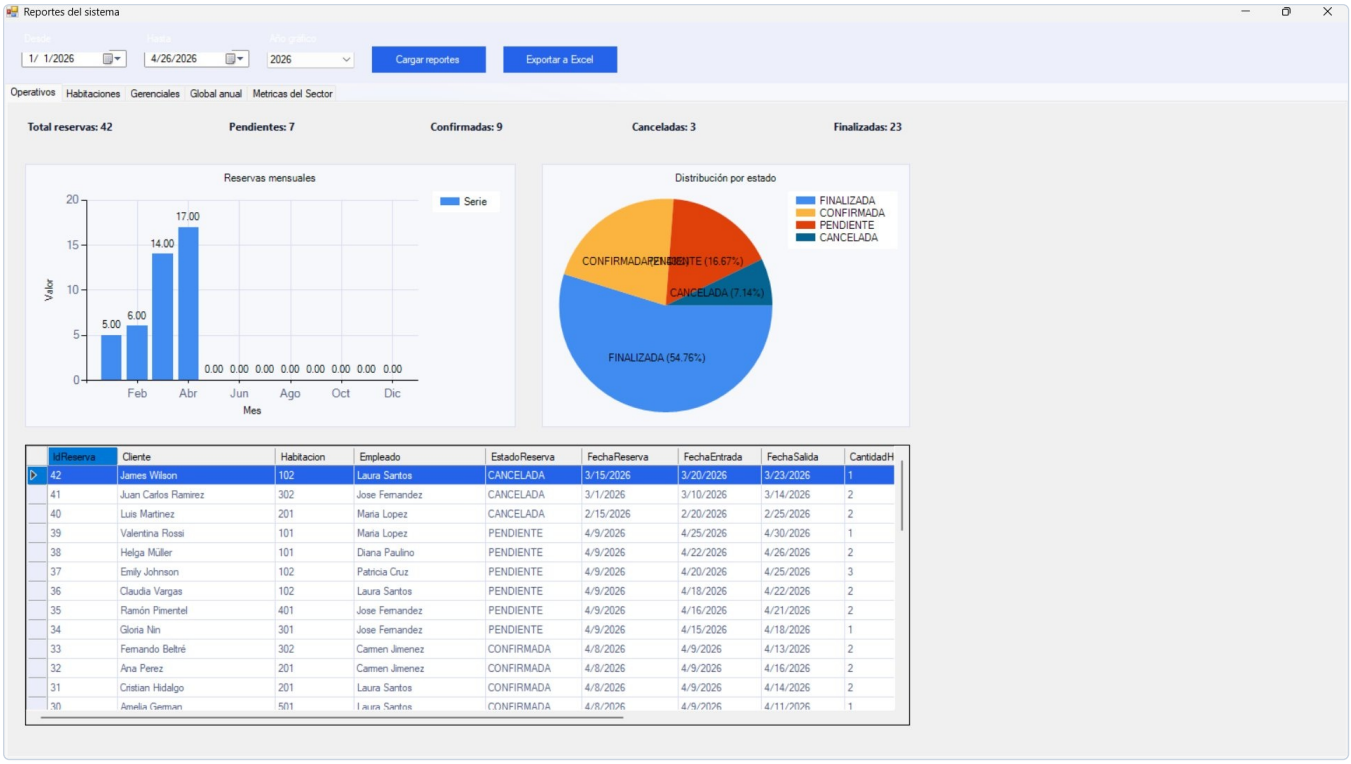
Módulo de habitaciones: filtrado por sucursal, tipo y estado; grilla con precio por noche y descripción.



Módulo de reservas: alta/edición, cálculo de noches y total, servicios adicionales y promociones aplicadas, con botones de Check-in y Check-out.

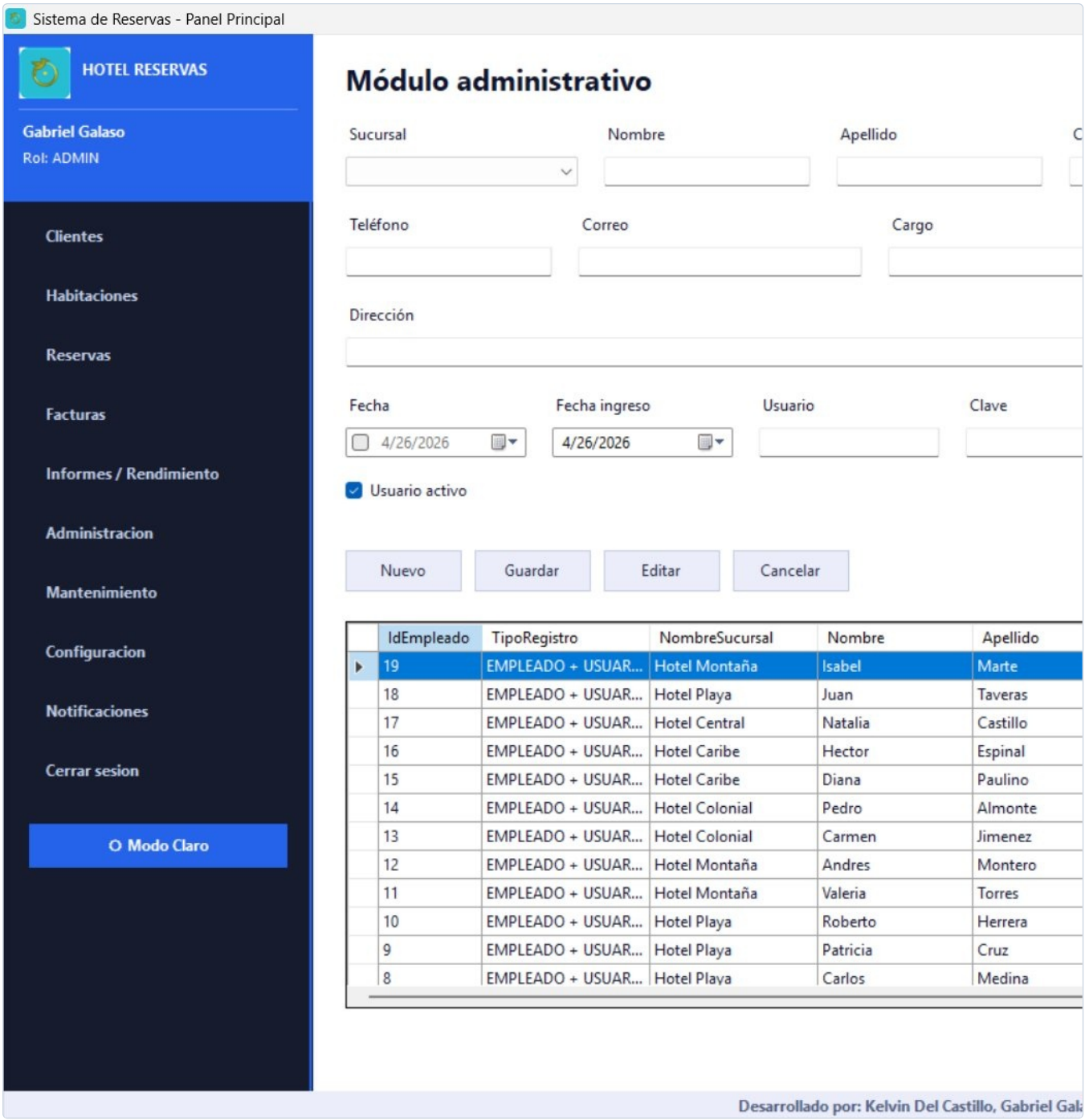


Módulo de facturas: cálculo automático de subtotal, descuento, impuesto y total; generación e impresión de factura, y registro de pagos.

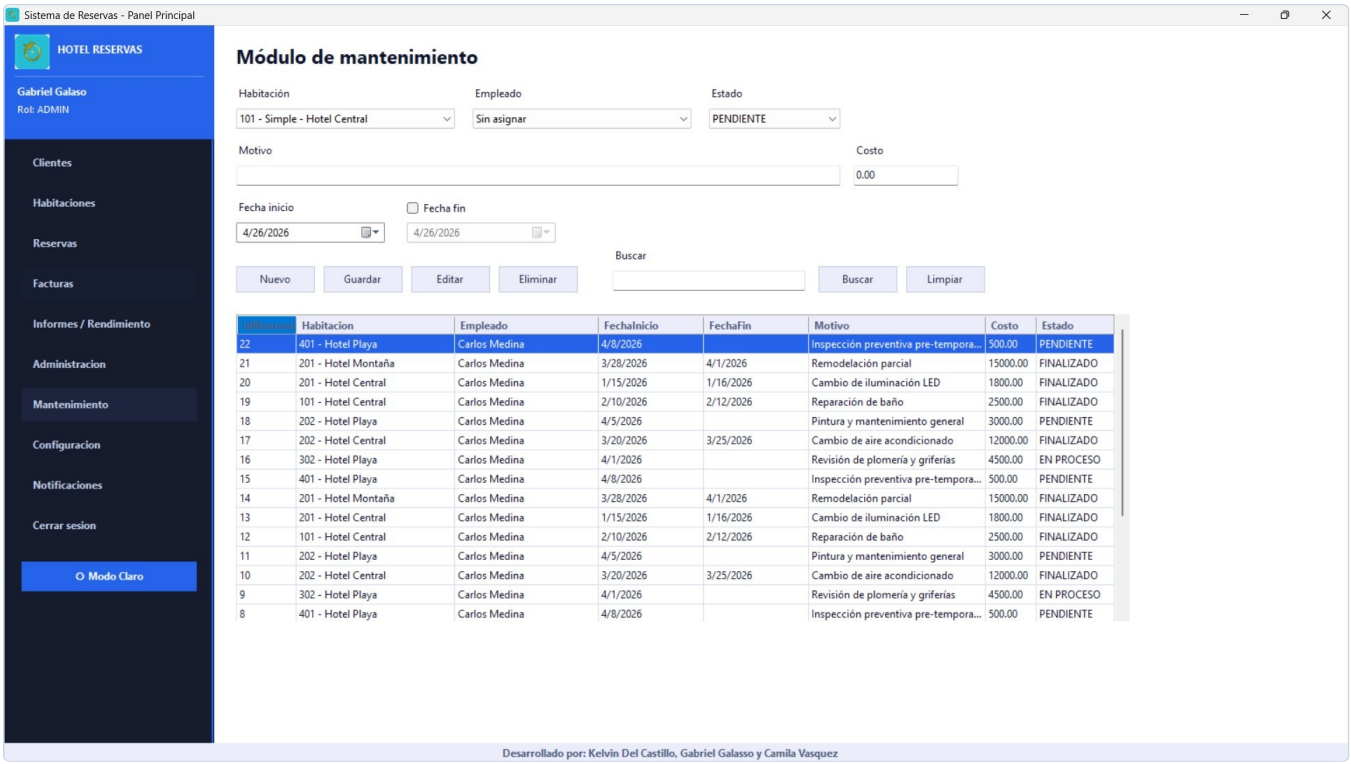


IdReserva	Cliente	Habitacion	Empleado	EstadoReserva	FechaReserva	FechaEntrada	FechaSalida	CantidadH
42	James Wilson	102	Laura Santos	CANCELADA	3/15/2026	3/20/2026	3/23/2026	1
41	Juan Carlos Ramirez	302	Jose Fernandez	CANCELADA	3/1/2026	3/10/2026	3/14/2026	2
40	Luis Martinez	201	Maria Lopez	CANCELADA	2/15/2026	2/20/2026	2/25/2026	2
39	Valentina Rossi	101	Maria Lopez	PENDIENTE	4/9/2026	4/25/2026	4/30/2026	1
38	Helga Müller	101	Diana Paulino	PENDIENTE	4/9/2026	4/22/2026	4/26/2026	2
37	Emily Johnson	102	Patricia Cruz	PENDIENTE	4/9/2026	4/20/2026	4/25/2026	3
36	Claudia Vargas	102	Laura Santos	PENDIENTE	4/9/2026	4/18/2026	4/22/2026	2
35	Ramón Pimentel	401	Jose Fernandez	PENDIENTE	4/9/2026	4/16/2026	4/21/2026	2
34	Gloria Nin	301	Jose Fernandez	PENDIENTE	4/9/2026	4/15/2026	4/18/2026	1
33	Fernando Beltré	302	Carmen Jimenez	CONFIRMADA	4/8/2026	4/9/2026	4/13/2026	2
32	Ana Perez	201	Carmen Jimenez	CONFIRMADA	4/8/2026	4/9/2026	4/16/2026	2
31	Cristian Hidalgo	201	Laura Santos	CONFIRMADA	4/8/2026	4/9/2026	4/14/2026	2
30	Amelia German	501	Laura Santos	CONFIRMADA	4/8/2026	4/9/2026	4/11/2026	1

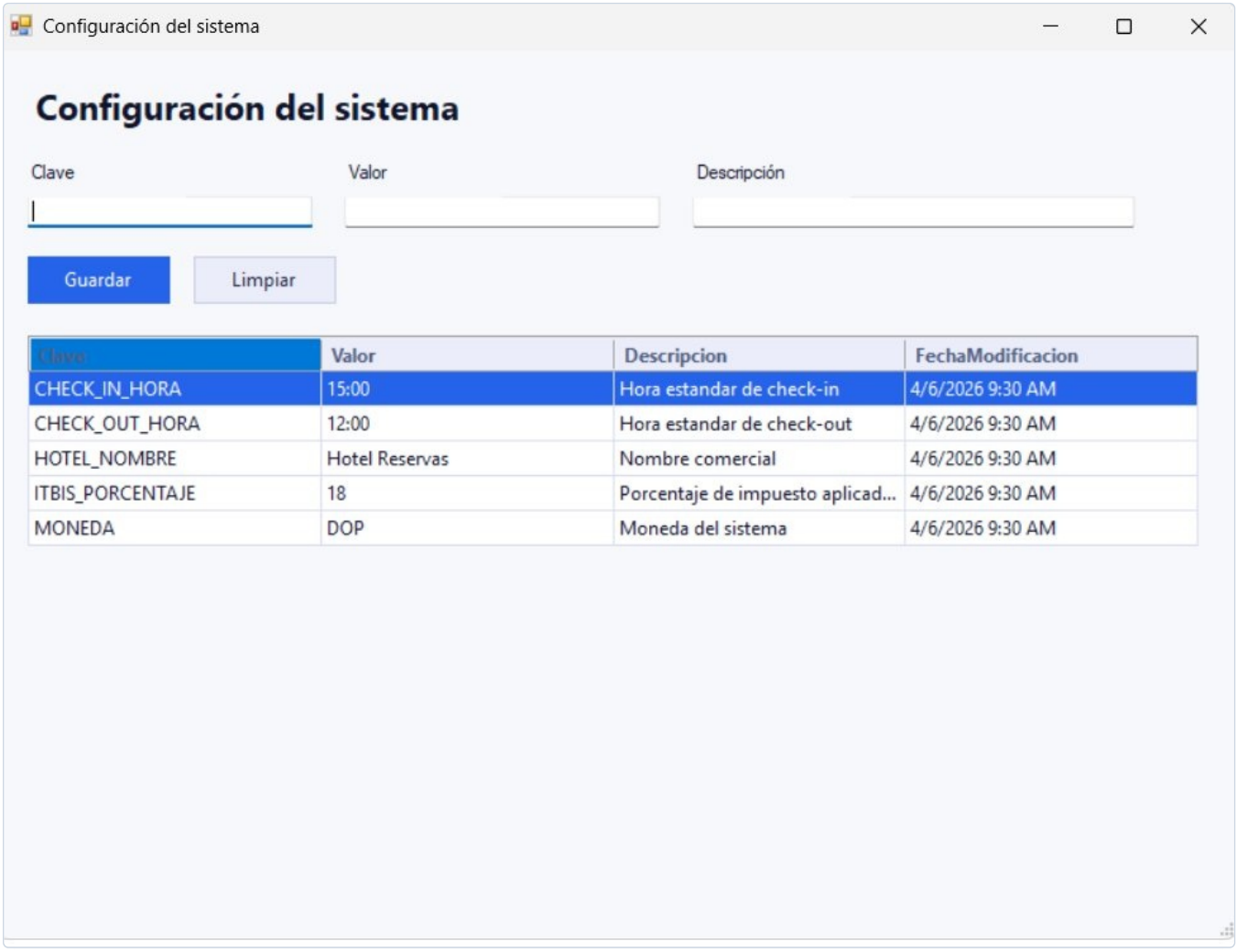
Reportes operativos: total de reservas por estado, gráfico de barras mensual, distribución por estado en gráfico de pastel y detalle tabular.



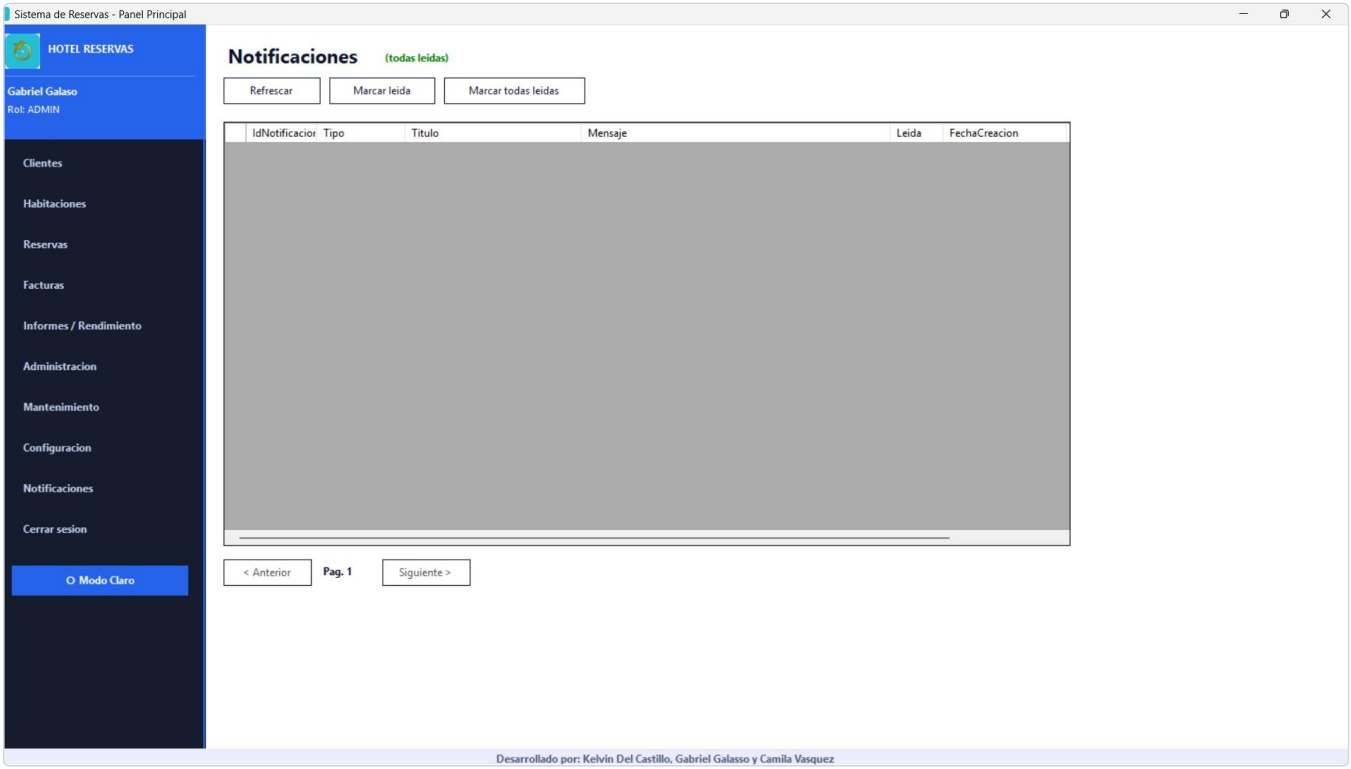
Módulo administrativo: gestión de empleados y usuarios con sucursal, rol y estado activo.



Módulo de mantenimiento: registro de mantenimientos por habitación con motivo, fechas, costo y estado (PENDIENTE / EN PROCESO / FINALIZADO).



Configuración del sistema: tabla clave-valor con parámetros como ITBIS_PORCENTAJE, MONEDA, CHECK_IN_HORA y CHECK_OUT_HORA.



Centro de notificaciones del usuario, con marcado individual o masivo y paginación.

Fin de la memoria.